Handling advanced modeling challenges Activity Book

Data Modeling for Qlik Sense

© 2023 QlikTech International AB. All rights reserved. All company and/or product names may be trade names, trademarks and/or registered trademarks of the respective owners with which they are associated.

DMQS May 2023

Table of Contents

Handling advanced modeling challenges Activity Book

Table of Contents

Activity 1: Order level aggregation

Challenge 3
Hints 3
Solution 4

Activity 2: Load quota data

Challenge 5
Hints 5
Solution 7

Activity 3 - Optional: Load the Bonus table

Challenge 7
Hints 8
Solution 10

Activity 4: Add the Sales vs. Quota sheet

Challenge 11
Hints 11
Solution 12

Activity 5 - Optional: Create the Link table

Challenge 13
Hints 13
Solution 16

Activity 6 - Optional: Bonus data

Challenge 16
Hints 17
Solution 18

Activity 7: Class function

Challenge 18
Hints 19
Solution 20

Activity 8 - Optional: If statements

Challenge 21

Hints 21
Solution 22

Activity 9: IntervalMatch function

Challenge 22
Hints 23
Solution 24

Activity 1: Order level aggregation

Summary:

- Create a new table from OrderDetails, using the Order level of granularity.
- Join the table to Orders.
- Load the data and review the new field in the model.

Qlik Sense app: ABC Sales Data

Challenge

In the **ABC Sales Data** app, create a new field that represents the total sales value for each order by doing the following:

- At the end of the **Orders** script section, add a **LOAD** statement to create the order-level table. The new table contains:
  - Two fields: *OrderID* and *Sum(LineSalesAmount)* **AS** *OrderSalesAmount*.
  - A **GROUP BY** clause at the end of the **LOAD** statement to aggregate the *OrderSalesAmount* field by *OrderID*.
  - Data from the *OrderDetails* table, by using a **RESIDENT LOAD** statement.
- **LEFT JOIN** the new table to the *Orders* table to add the *OrderSalesAmount* field.
- Load the data and preview the *Orders* table in the data model to verify that the *OrderSalesAmount* field exists.

Hints

- There is no need to add a label for the new table as it is joined to the *Orders* table.
- The location of the new table is significant. It must be created after the **LOAD** statement that creates the *LineSalesAmount* field, *OrderDetails*.

Step-by-step guide

1. Navigate to the end of the **Orders** script section and add the following statement:

```
//**** Aggregate LineSalesAmount to order level ****
LOAD
```

```
    OrderID,
    SUM(LineSalesAmount) AS OrderSalesAmount
RESIDENT OrderDetails
GROUP BY OrderID;
```

Copy to clipboard

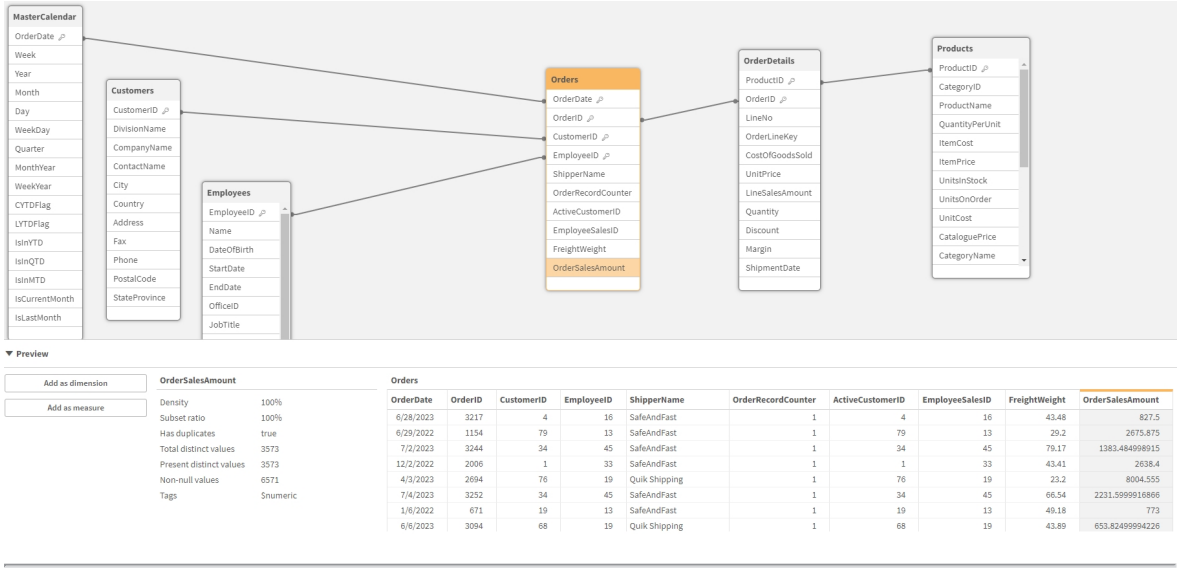
This statement aggregates the *LineSalesAmount* value from each line in the *OrderDetails* table over each occurrence of *OrderID*. The **GROUP BY** clause is needed to aggregate, or group, fields other than those included in aggregations. In this case, it totals the *LineSalesAmount* for each *OrderID*.

2. The result in this table needs to be added to the *Orders* table. To achieve this, **LEFT JOIN** the table to *Orders* by typing the following before the **LOAD** statement of this new table:

```
LEFT JOIN (Orders)
```

3. Load your data.
4. In the **Data model viewer**, preview the *Orders* table to check that the fields and data exist. The new *OrderSalesAmount* field simplifies some calculations and measures in sheet objects, such as the *% of sales over the total Order sale per LineNo*. This is achieved by using the *OrderID* and *LineNo* as dimensions and the following measure:
`Sum(LineSalesAmount) / Sum(OrderSalesAmount)`

Solution



Activity 2: Load quota data

Summary:

- Load quota data from an Excel source.
- Transform the table with the addition of a Crosstable prefix.

Qlik Sense app: ABC Sales Data

Challenge

In the **ABC Sales Data** app, load a new table as a cross table by doing the following:

- Examine the *Quota.xls* data file from the student collateral.
- Create a new script section after **MasterCalendar** and name it *Quota*.
- In the **Quota** section, load a new table, *Quotas*, using the *Quota.xls* data. Select only one of the sheets, *Quota 2022*. Make sure that *Embedded field names* is selected and the **Header size** is *1* row.
- In the *Quotas* table, comment out the *SalesPerson* field and add a new field:
  `'2022' AS QuotaYear,`
- Use the **CROSSTABLE** prefix to load the table:
  `CROSSTABLE(QuotaQuarter, QuotaAmount, 2)`
- Load the data and, in the data model, verify that the table has been loaded correctly.

Hints

- The quota data is held by employees and quarters for different years. There are two Excel worksheets in the *Quota.xls*, one per year, with the same table structure in each. The table is built as a cross table using *EmployeeID*, *SalesPerson*, and *Quarter*.

Step-by-step guide

1. Open and examine the *Quota.xls* file. This Excel file contains two worksheets, one for *2022* quotas and one for *2023* quotas. In each worksheet, there is a title and a table:
   - The title is not considered data. Therefore, do not load the first row of this worksheet in the app.
   - The table is a cross table because the quarter values are across the header. Qlik Sense will identify them as field names. The first two columns are presented as normal columns with the fields *EmployeeID* and *SalesPerson*. In this case, *SalesPerson* is the employee name and is not required as you have *EmployeeID* that associates with the *Employees* table.
2. Add a new script section following **MasterCalendar** named *Quota*.
3. Using the **Select data** option on the **DataFiles** connection, upload the *Quota.xlsx* file from the student collateral, and **select** it when the upload is complete.

   Qlik Sense on Windows: Use **Select data** on the **ExcelFiles** connection and select the *Quota.xls* file.
4. Select only one of the two tables, *Quota 2022*, and make sure all columns are selected.
5. The file contains one header row. Change the **Header size** to *1*. This updates the headers to the appropriate names.

6. The file also has embedded labels. Ensure that *Embedded field names* is selected in the **Select data** window.
7. Insert the script and comment out the field *SalesPerson*.
8. Create a new field below this one to insert the *QuotaYear,* as follows:

```
// SalesPerson,
'2022' AS QuotaYear,
```

9. Convert the table using the **CROSSTABLE** prefix: Add a new line above the **LOAD** statement, as follows:

```
CROSSTABLE(QuotaQuarter, QuotaAmount, 2)
```

In the table, *QuotaQuarter* is the **attribute** field containing the *Quarter* column heading converted. *QuotaAmount* is the **data** field containing the quota data converted. The number of **qualifier** fields is set to *2*, in this case, for *EmployeeID* and *QuotaYear*.

> **Note:** The number of qualifier fields is 2 and not 3 because the *SalesPerson* field has been commented out. The order of the fields in a cross table is important; the qualifier fields are defined as the first fields in the **LOAD** statement.

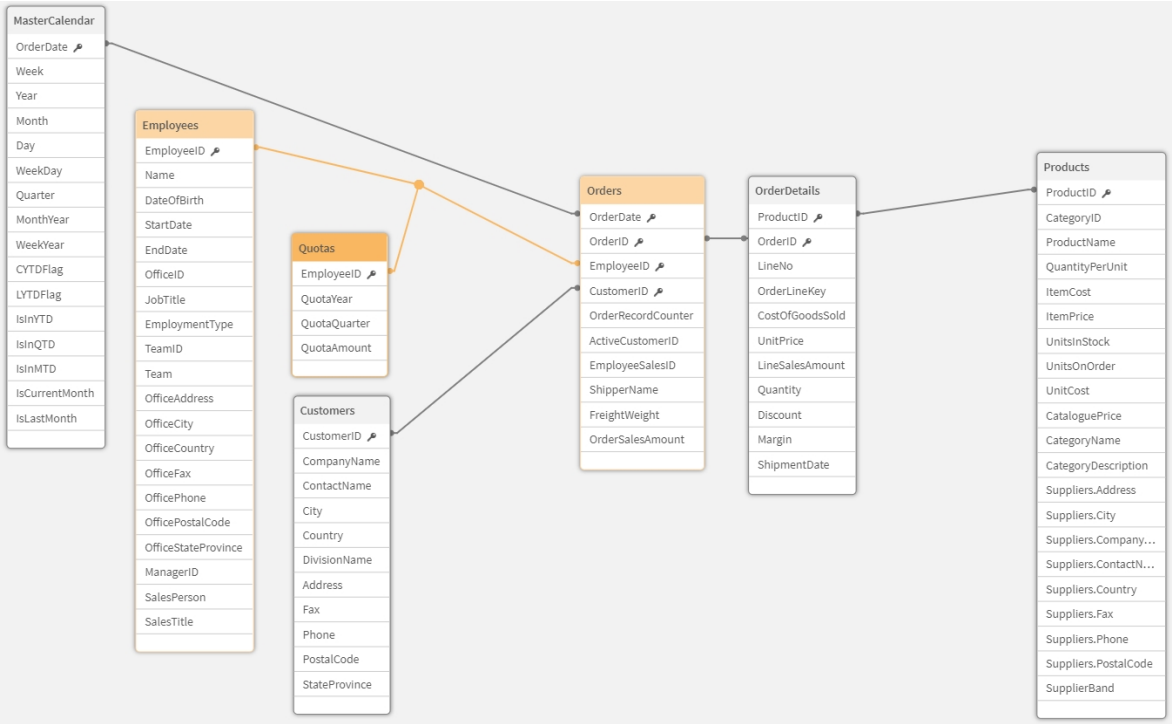
10. Label the new table *Quotas*. Your script should resemble the following:

```
Quotas:
CROSSTABLE(QuotaQuarter, QuotaAmount, 2)
LOAD
    EmployeeID,
// SalesPerson,
    '2022' AS QuotaYear,
    Q1,
    Q2,
    Q3,
    Q4
FROM [lib://<ConnectionName>/Quota.xls]
(biff, embedded labels, header is 1 lines, table is [Quota 2022$]);
```

Copy to clipboard

11. Load your data and check your data model. Verify that the new table *Quotas* exists and contains 4 fields.

Solution



Activity 3 - Optional: Load the Bonus table

Summary:

- Analyze the structure of the Bonus.xls file.
- Write a data load script to load and transform the Bonus data into a table.

Qlik Sense app: ABC Sales Data

Challenge

In the **ABC Sales Data** app, load the *Bonus* table from its Excel file by completing the following tasks:

- Examine the *Bonus.xls* file. Identify what needs to be done to load the table.
- At the end of the **Quota** section, load the *Bonus* table as a cross table and make sure it is associated with the data model.
- Use the load parameters and filters option for further transformation of your table. Perform the following:
  - Avoid loading the first row, as it is not part of the data.
  - Fill in the blank cells in the first column with the year value noted above them.

- Avoid loading the third column, *Quarter*, as it is blank and does not contain any information.

- Load the data and check the new table in the data model. Preview the table to verify that the values it contains are correct.

Hints

- The *Bonus* table represents the bonus data by *year*, *quarter*, and *employee*.
- In order for the *Bonus* table to be associated with the existing data model, the *EMPLOYEE_ID* must be renamed to *EmployeeID*.

Step-by-step guide

1. Open and examine the *Bonus.xls* file and review the table structure. Identify the changes needed to load this table and complete the load:
   - The first row in the Excel file is just a title and not part of the table.
   - This is a cross table. The quarter data is spread across columns.
   - The first column is incomplete. This needs addressing in the data load.
   - The third column is empty and contains no pertinent data for the table.
2. Using the **Select data** option on the **DataFiles** connection, upload the *Bonus.xlsx* file from the student collateral, and cancel the dialog when the upload is complete.

   Qlik Sense on Windows: The **ExcelFiles** connection already has access to the *Bonus.xls* file.

3. In the **Data load editor**, navigate to the end of the **Quota** section and add the following script (Do not forget to update the `[lib://...]` statement.):

```
Bonus:
CROSSTABLE(BonusQuarter, BonusAmount, 2)
LOAD
    YEAR AS BonusYear,
    EMPLOYEE_ID AS EmployeeID,
    Q1,
    Q2,
    Q3,
    Q4
FROM [lib://<ConnectionName>/Bonus.xls]
(biff, embedded labels, header is 1 lines, table is [Bonus$], filters
(Replace(1, top, StrCnd(null))));
```

   Copy to clipboard

   - The **CROSSTABLE** prefix combines the quarter data in one single column called *BonusQuarter*.
   - The *EMPLOYEE_ID* field has been renamed to *EmployeeID* to associate the *Bonus* table with the data model.

- The last parameter in the **FROM** clause specifies the filters applied to the data using a **filters** function. In this case, the only filter applied is **Replace**. It replaces the first column *(1)*, **YEAR**, with the previous value (in the row above), **top**, when the cell is blank or contains a null (**...,StrCnd(null)**).

Note: Another way of completing the *YEAR* field without using the **filters** function in the **FROM** clause is the following:

1. Create the *Bonus* table, as it is specified above. But, use *BonusTemp* for the table name and do not rename *YEAR*.

2. After the *BonusTemp* table, create a new table called *Bonus*. This table is created based on *BonusTemp* using a Resident Load.

3. In this new table, complete the *YEAR* field in a new *BonusYear* field using the **IsNull F** and **Peek F** functions in an **If F** statement.

4. Delete the *BonusTemp* table.

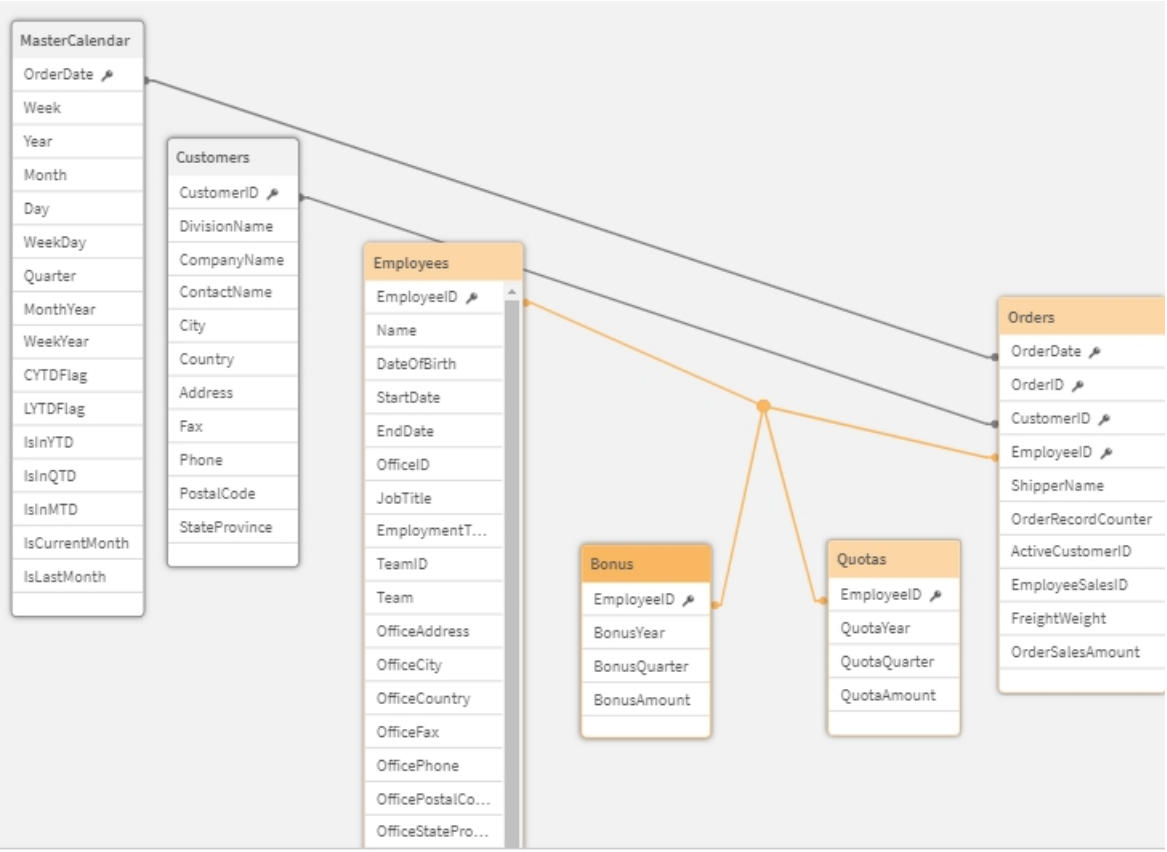
Using this method, the final script should appear as follows:

```
BonusTemp:
CROSSTABLE(BonusQuarter, BonusAmount, 2)
LOAD
    YEAR,
    EMPLOYEE_ID AS EmployeeID,
    Q1,
    Q2,
    Q3,
    Q4
FROM [lib://<ConnectionName>/Bonus.xls]
(biff, embedded labels, header is 1 lines, table is [Bonus$]);

Bonus:
LOAD
    EmployeeID,
    BonusQuarter,
    BonusAmount,
    IF(ISNULL(YEAR),PEEK(BonusYear), YEAR) AS BonusYear
RESIDENT BonusTemp;
DROP TABLE BonusTemp;
```

4. Load the data.
5. In your data model, select the *Bonus* table and preview its data to verify that the values are correct.

Solution



▼ Preview

Bonus	
Rows	168
Fields	4
Keys	1
Tags	$key $numeric $integer $ascii $text

Bonus

EmployeeID	BonusYear	BonusQuarter	BonusAmount
1	2023	Q1	10
1	2023	Q2	10
1	2023	Q3	10
1	2023	Q4	10
8	2023	Q1	20
8	2023	Q2	20
8	2023	Q3	25
8	2023	Q4	30

Activity 4: Add the Sales vs. Quota sheet

Summary:

- Create a new sheet to analyze Sales and Quotas measures.
- Identify issues between the known source data and the data as the model presents it.

Qlik Sense app: ABC Sales Data

Challenge

In the **ABC Sales Data** app, create a new sheet to show the *Sales vs. Quota* trend by *Quarter* and *Year* and observe the wrong results by completing the following tasks:

- In the data model, examine the tables *Quotas* and *Orders*.
- Create a new app sheet named *Sales vs. Quota* and add a **Line chart** with:
  - *Year* as the dimension.
  - *Sum(LineSalesAmount)* as *Sales*, for the measure.
  - *Sum(QuotaAmount)* as *Quota*, for the second measure.
- Add the fields *Year*, *Quarter*, *QuotaYear*, *QuotaQuarter*, *Name*, and *EmployeeID* as **Filter panes**.
- Analyze the **Sales vs. Quota** sheet and make your observations.

Hints

- The issues identified in this activity make a Link table solution necessary.

Step-by-step guide

1. In the data model, analyze the fields in the *Quotas* and *Orders* tables.

   Note: The *Quotas* table contains the *QuotaAmount* per *EmployeeID* over the *QuotaYear* and *QuotaQuarter*. The *EmployeeID* field is the key field in this table and relates the table with the *Orders* table. The *Orders* table is associated with the *OrderDetails* table that contains the *LineSalesAmount* field used to calculate the *Sales* measure.
   The *Orders* table also contains the *OrderDate* field that associates this table with the *MasterCalendar* and the *Year* and *Quarter* fields.

2. Within **Analyze** mode, create a new sheet and name it *Sales vs. Quota*.
3. In sheet edit mode, insert a **Line chart** to fill the top half of the sheet.
4. Add the *Year* field as the dimension and the *Sum(LineSalesAmount)* as the measure. Name the new measure *Sales*.
5. From **Fields** in the **Assets Panel**, add a second measure by dragging the *QuotaAmount* onto the chart. Select *Sum(QuotaAmount)* for the aggregation and name the new measure *Quota*.
6. In the free space of the sheet, add the following fields as **Filter panes**: *Year*, *Quarter*, *QuotaYear*, *QuotaQuarter*, *Name*, and *EmployeeID*.
7. Disable the edit mode and analyze the sheet.
   Note that the line chart is not correct:
   - The line representing the *Quota.* is displayed across all the years in the chart, but remember the *Quotas* table just has two *QuotaYear* values.

- There is a *QuotaAmount* with no year (*null* value) in the chart.
- Select *Bob Park* in the *Name* field and note that the line representing the *Quota* goes flat across the timeline.

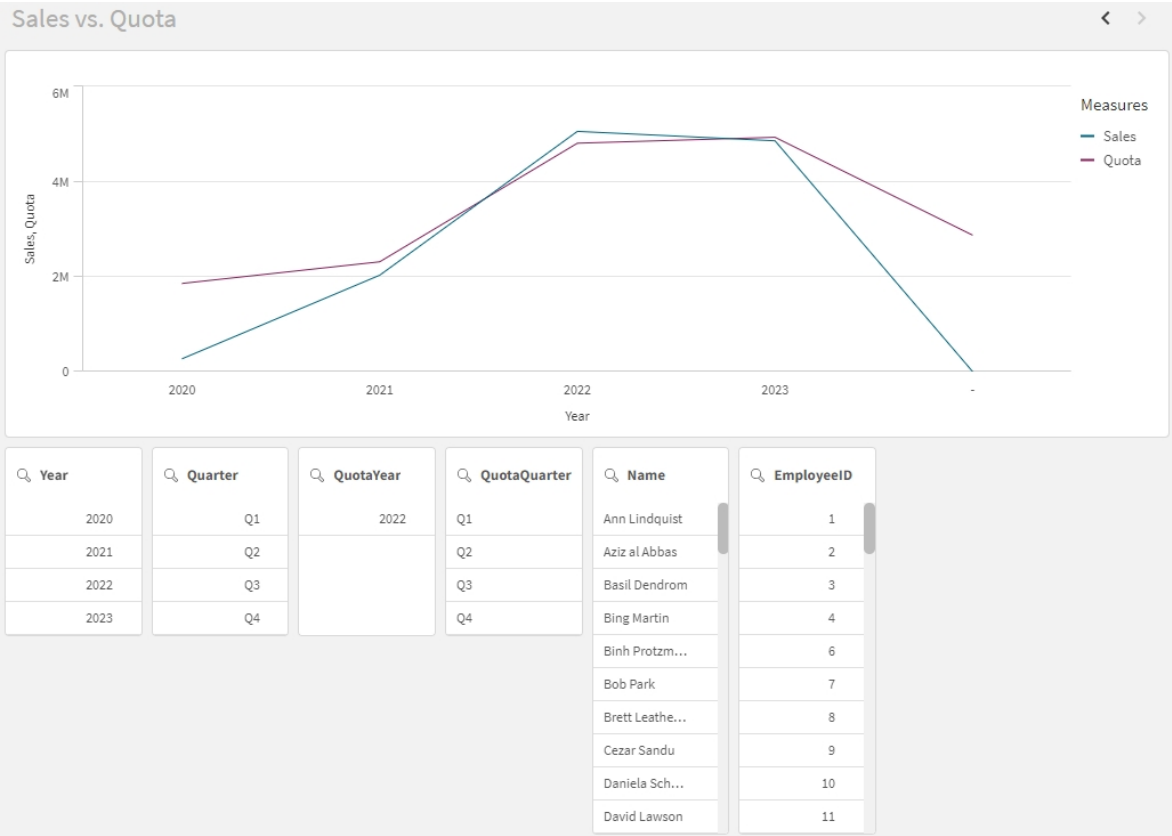
> **Note:** The problem lies in the associative data model. The *Quota* table associates with the rest of the tables in the data model using only *EmployeeID*. The *Year* field, used as a dimension in the chart, comes from the *MasterCalendar* table that is associated with the rest of the data model using the *Orders* table by the *OrderDate* field.

Based on this, the red line representing the *Quota* is:

- Summing up the quotas over all the quota years, per each year, in the chart. The difference between years is due to the different employees' sales activities.
- Displaying a total quota amount, not associated with any year (*null* value) in the chart dimension. There are employees with *Quota*, but no *Sales*, like *Erik Fredriksen*.

8. Clear your selections, if there are any.
9. Consider potential solutions.

Solution



Activity 5 - Optional: Create the Link table

Summary:

- Create concatenated keys within the Quotas and Orders tables, using the fields EmployeeID, Year, and Quarter.
- Generate a link table between Orders and Quotas to maintain correct associations and provide a bridge to Employees.

Qlik Sense app: ABC Sales Data

Challenge

In your **ABC Sales Data** app, create a Link table to resolve the issue in the line chart from the **Sales vs. Quota** sheet by doing the following:

- Navigate to the **Quota** script section and rename the *QuotaYear* to *Year* and the *QuotaQuarter* to *Quarter*.
- Load the data and observe the circular reference in your data model. Try to identify the fields and tables that cause the problem.
- Rename the *Quotas* table to *QuotasTemp*, and rename the above fields back to their original values. Alias the field *EmployeeID* to *QuotaEmployeeID*.
- Create the main *Quotas* table that contains all the fields from *QuotasTemp* and the composite field named *EmpYrQtr_Key*.
- In the **Orders** section, locate the *Orders* table and alias *EmployeeID* as *OrdersEmployeeID*.
- Use a preceding **LOAD** on preceding **LOAD** to create the composite key as above. Use the same fields and the same key name.
- Remove the fields, *Year* and *Quarter,* from the *MasterCalendar* table.
- Create a new script section after **Quota** and name it *LinkTable*. Create the script for a Link table according to the steps in the theory section.
- At the end of the **LinkTable** script section, drop the fields *QuotaYear, QuotaQuarter* and *QuotaEmployeeID,* as you do not need them any more.

Hints

- A Link table is necessary in order for the **Sales vs. Quota** sheet to produce correct results.
- A synthetic key is caused when associating the *QuotaYear* and *QuotaQuarter* fields with the *Year* and *Quarter*. This synthetic table is resolved with the Link table.

Step-by-step guide

1. A change in the data model is required at this stage. In the **Data load editor**, locate the **Quota** section.
2. The *QuotaYear* and the *QuotaQuarter* need to be renamed to make sure the *Quotas* table associates to the data model correctly. Therefore, replace the *QuotaYear* and the *QuotaQuarter* fields in the *Quotas* table with *Year* and *Quarter*, respectively.
3. Load the data and notice that a circular reference warning appears during the script execution.

4. Analyze the circular reference in the data model.

> **Note:** A **synthetic table**, *$Syn 1 Table* has been created between *MasterCalendar* and *Quotas,* as they have now two fields with the same name, *Year* and *Quarter*.
> There is also a **circular reference** between the *Quotas*, *MasterCalendar*, *Orders* and *Employees* tables. The *Orders* table has been disconnected from the data model due to this loop and is now a **loosely coupled** table.
> In this case, you cannot rename fields or delete them, as they are all required as part of a compound key. A **Link table** must be created between the *Quotas*, *MasterCalendar*, *Orders,* and *Employees* tables, with all the common fields and keys to associate them.

5. In the **Data load editor**, locate the **Quota** script section and change the *Quotas* table name to *QuotasTemp*.
6. In the *QuotasTemp* table, rename the *Year* and *Quarter* fields back to *QuotaYear* and *QuotaQuarter*. Also, alias *EmployeeID* as *QuotaEmployeeID*. This breaks the association between this table and the rest of the data model.
7. After the *QuotasTemp* table **LOAD** segment, add the following script code:

```
Quotas:
LOAD
    *,
    QuotaEmployeeID & '|' & QuotaYear & '|' & QuotaQuarter AS
EmpYrQtr_Key
RESIDENT QuotasTemp;

DROP TABLE QuotasTemp;
```

Copy to clipboard

This **LOAD...RESIDENT** statement creates the key field in the *Quotas* table.

8. In the **Orders** script section, alias the *EmployeeID* field to *OrdersEmployeeID,* in the *Orders* table.
9. Create the *EmpYrQtr_Key* field in the *Orders* table using the following preceding **LOAD** statement:

```
LOAD
    *,
    OrdersEmployeeID & '|' & YEAR(OrderDate) & '|' & 'Q' & CEIL(MONTH
(OrderDate) / 3) AS EmpYrQtr_Key;
```

Copy to clipboard

10. In the **MasterCalendar** section, comment out the *Year* and *Quarter* fields from the *MasterCalendar* table.
    The fields are not really needed in the *MasterCalendar* table, as they will be part of the Link table.
11. Create a new script section named *LinkTable*, after the **Quota** section, and add the following script:

```
LinkTable:
LOAD DISTINCT    // *************** From Quotas table
    EmpYrQtr_Key,
    QuotaEmployeeID AS EmployeeID,
    QuotaYear AS Year,
    QuotaQuarter AS Quarter,
    'Quotas' AS FromTable
RESIDENT Quotas;

CONCATENATE (LinkTable)
LOAD DISTINCT    // *************** From Orders table
    EmpYrQtr_Key,
    OrdersEmployeeID AS EmployeeID,
    YEAR(OrderDate) AS Year,
    'Q' & CEIL(MONTH(OrderDate) / 3) AS Quarter,
    'Orders' AS FromTable
RESIDENT Orders;
```

Copy to clipboard

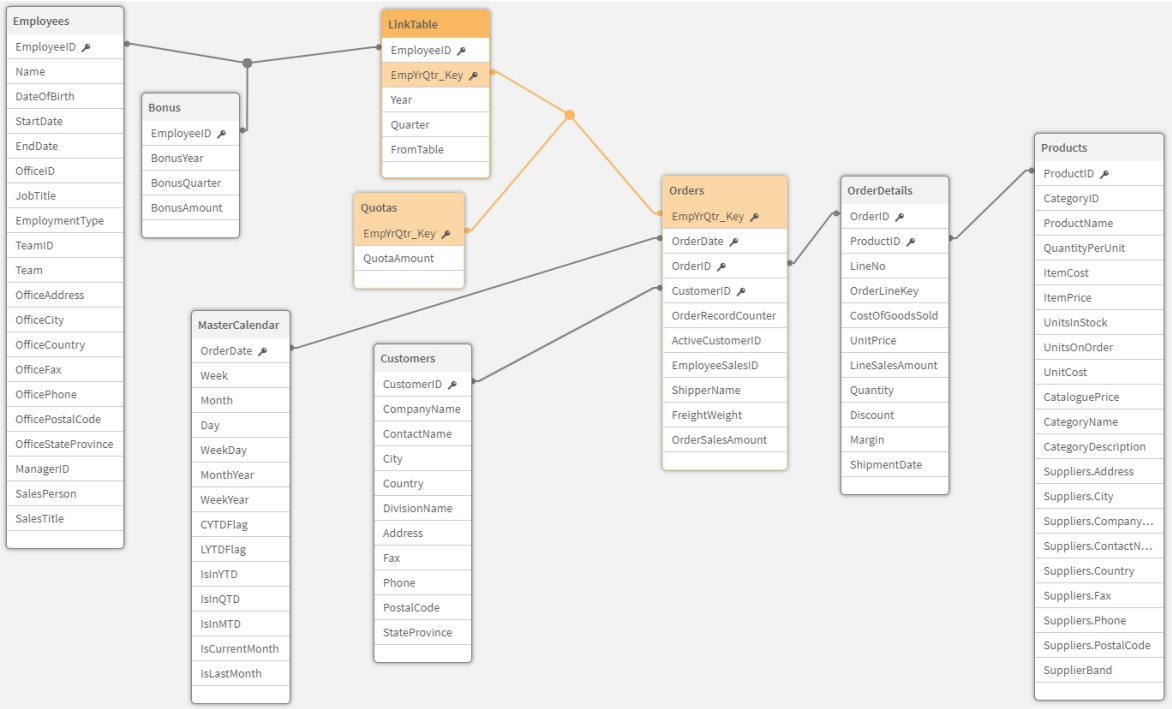
Note that a Link table is created using a **LOAD... RESIDENT** statement from the tables with common dimensions. **LOAD DISTINCT** is used to avoid repetition in the data and the **CONCATENATE** prefix is used to unify the tables with all the possible combinations of values for the dimensions. There is a new field called *FromTable* that helps to identify the tables from which each record is located. This field is optional.

12. Delete unneeded fields by adding this statement at the end of the **LinkTable** section:

```
DROP FIELDS QuotaYear, QuotaQuarter, QuotaEmployeeID, Order-
sEmployeeID;
```

13. Load the data and verify that your data model resembles the **Solution** image.
14. Return to the **Sales vs. Quota** app sheet and review the line chart. Even though the chart looks better, there is still something you can do to improve it. The chart lines currently show zeros for the data points where data does not exist. In the sheet edit mode, edit the line chart **Properties**; Replace the **Quota** measure expression with the following:
    `IF(SUM(QuotaAmount)=0, Null(), SUM(QuotaAmount))`
15. Change the line chart style to *Area* (**Appearance** > **Presentation**).
16. The results should now be correct. Also, notice how the filters display an invalid dimension error message, as these fields were deleted. You can safely remove them from your sheet.

Solution



Activity 6 - Optional: Bonus data

Summary:

- Create a new sheet and table to analyze discrepancies with the Bonus data.
- Amalgamate the Bonus data with the existing Link table to address the issues.

Qlik Sense app: ABC Sales Data

Challenge

In the **ABC Sales Data** app, create a new sheet to analyze *bonus* data vs. *quota* and *sales*. Identify any issues and work towards their resolution by completing the following tasks:

- Create a new sheet named *Bonus* and add a **Table** with the following fields:
  - *Year*, *Quarter*, and *SalesPerson* as the dimensions.
- Add the following as measures:
  - *Sales: Sum(LineSalesAmount)*
  - *Quota: Sum(QuotaAmount)*
  - *Target%: Sales / Quota*
  - *Bonus: Sum(BonusAmount)*
  - *Extra: (Bonus / Target) * 100*

- Sort the table by *Target%* descending.
- Analyze the data and check if the results are correct.
- As a first step to resolving the wrong results in the *Bonus* table script, rename the *BonusYear* field to *Year* and *BonusQuarter* field to *Quarter*.
- Load the data and analyze the outcome. Consider how to eliminate the synthetic key that appears in your data model.

Hints

- A Link table is required to eliminate the synthetic key between the *Bonus* table and the *LinkTable*.
- The process to create the Link table is similar to the *Quotas* Link table creation.

Step-by-step guide

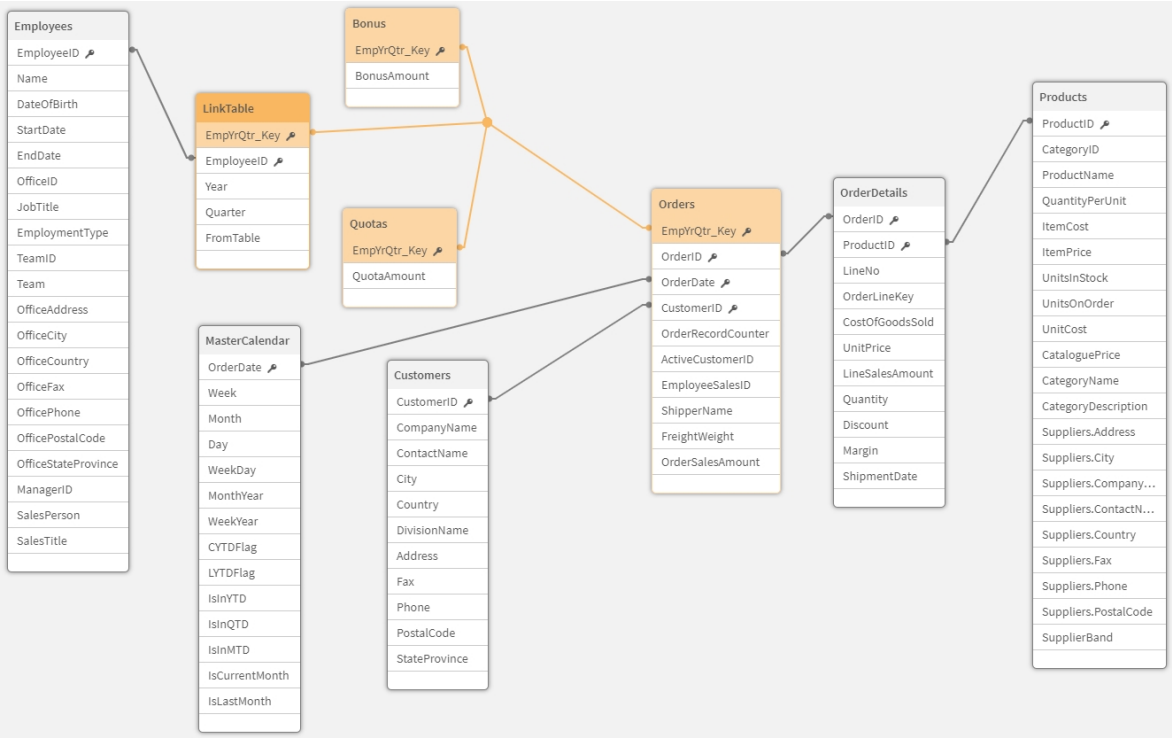
1. In **Analyze** mode, create a new sheet named *Bonus*.
2. In sheet edit mode, drag a **Table** object onto the sheet.
3. Add *Year*, *Quarter*, and *SalesPerson* as the dimensions.
4. Add the following expressions as measures to the table and sort by *Target%* descending:

Measure	Label	Format
`Sum(LineSalesAmount)`	*Sales*	Number
`Sum(QuotaAmount)`	*Quota*	Number
`Sales / Quota`	*Target%*	Number%
`Sum(BonusAmount)`	*Bonus*	Number
`(Bonus * Target%) * 100`	*Extra*	Number

5. Notice the numbers in the *Bonus* column. Are these results correct when comparing with the *Bonus* Excel file? If there is a problem, how would you resolve it?
6. In the **Data load editor**, navigate to the **Quota** script section and locate the *Bonus* table. Rename the *BonusYear* field to *Year* and *BonusQuarter* field to *Quarter*.
7. Load your data and notice that a new synthetic table is created. Analyze the results and consider possible solutions.
8. See the **Solution** image for the recommended next steps.

> **Note:** In order to transform the data model to resemble the **Solution** image, the *Bonus* table needs to be transformed further. It should associate with the *LinkTable,* following the same process completed for the *Quotas* table.

Solution



Activity 7: Class function

Summary:

- Create a new derived field to hold price groups of product UnitPrice.
- Analyze the new field in a table object.

Qlik Sense app: ABC Sales Data

Challenge

In the **ABC Sales Data** app, use the **CLASS** function to create intervals based on a field or an expression. You can achieve this by doing the following:

- In the **Products** script section, add a **CLASS** statement to create the *PriceGroup* field after the *CataloguePrice* field.
- The **CLASS** function creates intervals of *100*, called *CataloguePrice*, using the *UnitPrice* values.
- Load the data and preview the *Products* table in the data model. Verify that the new field has been added.

- Create a new app sheet named *Price Groups* and add a **Table** with the following columns:
  - *PriceGroup* as the dimension.
  - *Count(ProductName)* as *Products* for the measure.
- Use the **REPLACE** function to create a shorter dimension label that resembles the following:
  `>= 100 < 200`
- Load the data and navigate to the **Price Groups** sheet to see the result.

Hints

- The **CLASS** function only creates intervals of the same size.
- **CLASS** is both a script and a chart function.

Step-by-step guide

1. Select the **Products** script section and add the following line to the **LOAD** statement for *Products*:

```
CLASS(UnitPrice, 100, 'CataloguePrice') AS PriceGroup,
```

2. Load the data and, in the data model, preview the *Products* table. A new field for *PriceGroup* has appeared containing intervals of *100* in the form:
   `100 <= CataloguePrice < 200`
3. Navigate to the **Analyze** mode, and create a new sheet named *Price Groups*.
4. In the sheet edit mode, add a **Table** to the new sheet, spanning 4 rows and 8 columns, with the field *PriceGroup* as the dimension and the expression *Count(ProductName)* as the measure. Label the measure *Products*.
5. Disable the sheet edit mode and analyze the data. The table shows the price groups and the corresponding numbers of products in each group.

   Note: A visible problem here is that the price group labels are long text strings which uses too much space when displayed in some charts. Using the **Replace F** function will help shorten this label.
   The syntax of the **Replace** function is the following:
   `Replace (string, 'old substring' , 'new substring' )`

6. The dimension label that the original class expression returns is: *100 <= CataloguePrice < 200*. To make it shorter, replace the following part of the string '<= *CataloguePrice* <'.
   In addition, the bucket label *100 – 200* is not quite correct due to the <= and < operators. The bucket, for this example, actually contains values from *100* to *199.99*. Use the **REPLACE** function to change the substring for a < symbol:
   `100 < 200`
   You can also concatenate using a '>=' substring to clarify that the bucket contains values from 100 inclusive:
   `>= 100 < 200`
   Therefore, in the **Products** script section, edit the script for *PriceGroup* as follows:

```
'>= ' & REPLACE(CLASS(UnitPrice, 100, 'CataloguePrice'), '<= Cata-
loguePrice <', ' < ') AS PriceGroup,
```

Copy to clipboard

7. Load the data and review the *Products* table in the data model. The *PriceGroup* now contains intervals in the form:
   `>= 100 < 200`
8. In the **App overview**, open the **Price Groups** sheet and verify that it shows the price groups with their updated and shorter labels.

Solution

Price Groups

PriceGroup	Products
Totals	78
>= 0 < 100	22
>= 100 < 200	8
>= 200 < 300	7
>= 300 < 400	10
>= 400 < 500	8
>= 500 < 600	3
>= 600 < 700	3
>= 700 < 800	4
>= 800 < 900	4
>= 900 < 1000	6
>= 1300 < 1400	1
>= 1500 < 1600	1
>= 1600 < 1700	1

Activity 8 - Optional: If statements

Summary:

- Re-work the previous PriceGroup expression to utilize nested IF statements.
- Analyze the resulting data classification field within the existing table object.

Qlik Sense app: ABC Sales Data

Challenge

In the **ABC Sales Data** app, use the **IF** function to create groupings, like **CLASS**, but with varying sizes of buckets. You can achieve that by doing the following:

- In the **Products** script section, comment out the existing *PriceGroup* field.
- Create a new field called *PriceGroup.* This time, use nested **IF** statements.
- Make sure the **IF** statements create four groups based on the *UnitPrice* value:
  - >= 0 < 100
  - >= 100 < 500
  - >= 500 < 1000
  - >= 1000
- Load the data and check your data model to verify the field has been created.
- Navigate to the **Price Groups** sheet and check the results.

Hints

- The **IF** statement refers to the *UnitPrice*, and not the *CataloguePrice* field, because it is created in the same **LOAD** where *CataloguePrice* is defined.
- This approach can prove to be complex and difficult to maintain if there are many different group ranges.

Step-by-step guide

1. In the **Data load editor**, navigate to the **Products** script section and locate the *Products* table.
2. Comment out the *PriceGroup* field you created earlier. Add the following script to a new line, after the commented field:

   ```
   IF(UnitPrice>=0 AND UnitPrice<100,'>= 0 < 100',
   ```

   This is the first part of the **IF** statement. If the product meets the condition (*UnitPrice* between 0 and 100), the *then* part of the expression is used and the result is the value of *>= 0 < 100*. If the product does not meet the condition, the *else* portion is used.
3. For the *else* portion, start a new nested **IF** statement with a new condition. The script continues with several nested **IF** statements. Complete the statement so it reads as follows:

   ```
   IF(UnitPrice>=0 AND UnitPrice<100,'>= 0 < 100',
       IF(UnitPrice>=100 AND UnitPrice<500,'>= 100 < 500',
           IF(UnitPrice>=500 AND UnitPrice<1000,'>= 500 < 1000',
               IF(UnitPrice>=1000,'>= 1000')))) AS PriceGroup,
   ```

   Copy to clipboard
4. Load the data and preview the *Products* table in the data model to see the new intervals.
5. In the **App overview**, open the **Price Groups** sheet and make sure that the table is sorted alphabetically, by *PriceGroup* ascending.

Solution

Price Groups

PriceGroup	Products
Totals	78
>= 0 < 100	22
>= 100 < 500	33
>= 500 < 1000	20
>= 1000	3

Activity 9: IntervalMatch function

Summary:

- Re-engineer the PriceGroup expression to leverage an IntervalMatch construct.
- Assemble a temporary, inline table to hold the value boundaries.
- Ensure the final result is the same as prior data classification techniques.

Qlik Sense app: ABC Sales Data

Challenge

In the **ABC Sales Data** app, use the **IntervalMatch** function to create a price grouping for each product's unit price. To accomplish this, complete the following tasks:

- In the **Products** script section, comment out the field *PriceGroup*.
- Create a new script section before **LoadComplete** and name it *IntervalMatch*.
- Create an **INLINE** table with three columns: *Start*, *Stop,* and *PriceGroup.* Name the table *PriceGroup.*
- Create four groups in this table, by defining the *Start*, *Stop,* and *PriceGroup* label value for each group, as below:
  - >= 0 < 100
  - >= 100 < 500
  - >= 500 < 1000
  - >= 1000
- Create the **IntervalMatch** table to match the *CataloguePrice* values from the four groups.
- **LEFT JOIN** the new table to the *PriceGroups* table to eliminate the synthetic key.
- Drop the fields *Start* and *Stop* as they are no longer needed.

- Load the data and verify there are no issues in the data model.
- Navigate to the **Price Groups** sheet to check the results.

Hints

- The price group values can be kept to an external source, meaning that there is no need to manipulate the script every time someone needs to modify the group limits.
- When the **IntervalMatch** function is used, a synthetic key is always created. Joining the table is the usual way of eliminating this synthetic key.

Step-by-step guide

1. Navigate to the **Products** script and comment out the entire set of nested **IF** statements relating to the *PriceGroup* field.
2. Add a new section before the **LoadComplete** section named *IntervalMatch*.
3. Create an **INLINE** table to store the price group intervals.

   Note: An **Inline** table is a table that is created entirely in the script along with its data in the same statement.

   Type the following statement into the **IntervalMatch** section:

   ```
   // **** Load product groups as inline table ****
   PriceGroup:
   LOAD * INLINE [
       Start, Stop, PriceGroup
       0,     100,  >= 0 < 100
       100,   500,  >= 100 < 500
       500,   1000, >= 500 < 1000
       1000,  ,     >= 1000
   ];
   ```

   Copy to clipboard

   This script creates a new table named *PriceGroup*, with interval data, using an **INLINE LOAD** statement. The structure inside the square brackets contains the table information. The first row has three column headings. The subsequent rows are data rows, also with three columns of data.

   Note: Notice that the end of each line does not end with a comma. All row data must be on the same line with each *value* separated by a comma.

4. To create an **IntervalMatch** with *CataloguePrice* and your **INLINE** table, type the following script after the *PriceGroup* table:

   ```
   INTERVALMATCH(CataloguePrice)
   LOAD
       Start,
       Stop
   RESIDENT PriceGroup;
   ```

Copy to clipboard

5. Load the data and preview the data model: The *PriceGroup* table is your **INLINE** table. The *PriceGroup-10* table is the result of the **IntervalMatch** statement. The *$Syn 1 table* is the synthetic table created because both tables contain the *Start* and *Stop* fields.

> **Note:** The **IntervalMatch** method will always lead to a synthetic key in your data model. The standard process to eliminate this synthetic key is to **Left Join** the new **IntervalMatch** table to the table that contains the group definitions.

6. You must now eliminate the synthetic table by joining *PriceGroup-10* into *PriceGroup* with a **LEFT JOIN**. Add the **LEFT JOIN** statement between **INTERVALMATCH** and **LOAD** as follows:

```
INTERVALMATCH(CataloguePrice)
LEFT JOIN (PriceGroup)
LOAD
    Start,
    Stop
RESIDENT PriceGroup;
```

7. Load the data and verify that the data model does not contain any synthetic keys.
8. Remove the *Start* and *Stop* fields with a **DROP FIELDS** statement. Add this statement at the end of the **IntervalMatch** section as follows:

```
DROP FIELDS Start, Stop;
```

9. Load the data again. Within **Analyze** mode, open the **Price Groups** sheet. Make sure the table is sorted by *PriceGroup*, in ascending, alphabetical order.

> **Note:** After it is created, the **IntervalMatch** solution is the easiest to maintain while still allowing flexibility for varying intervals. The **Inline Load** from this activity, has been used only for the scope of this module. In reality, the **Inline Load** would be replaced by data from an external source, meaning that interval ranges can be set by other systems or modified by authorized users.

Solution

Price Groups

PriceGroup	Products
Totals	78
>= 0 < 100	22
>= 100 < 500	33
>= 500 < 1000	20
>= 1000	3